

APTPU: Approximate Computing Based Tensor Processing Unit

Mohammed E. Elbity¹, *Student Member, IEEE*, Peyton S. Chandarana, Brendan Reidy,

Jason K. Eshraghian², *Member, IEEE*, and Ramtin Zand¹

Abstract—We propose an approximate tensor processing unit (APTPU), which includes two main components: (1) approximate processing elements (APEs) consisting of a low-precision multiplier and an approximate adder, and (2) pre-approximate units (PAUs) which are shared among the APEs in the APTPU's systolic array, functioning as the steering logic to pre-process the operands and feed them to the APEs. We conduct extensive experiments to evaluate the performance of the APTPU across various configurations and various workloads. The results show that the APTPU's systolic array achieves up to $5.2 \times \text{TOPS}/\text{mm}^2$ and $4.4 \times \text{TOPS}/\text{W}$ improvements compared to that of a conventional systolic array design. The comparison between the proposed APTPU and in-house TPU designs shows that we can achieve approximately $2.5\times$ and $1.2\times$ area and power reduction, respectively, while realizing comparable accuracy. Finally, a comparison with the state-of-the-art approximate systolic arrays shows that the APTPU can realize up to $1.58\times$, $2\times$, and $1.78\times$ reduction in delay, power, and area, respectively, while using similar design specifications and synthesis constraints.

Index Terms—Approximate computing, tensor processing units, machine learning hardware accelerator, systolic array.

I. INTRODUCTION

DEEP learning workloads demand high resource utilization and frequent memory accesses, and thus require well-engineered memory bandwidth optimization. Neural networks are an optimal fit for architectures that perform parallel computing with a deeply pipelined network of processing elements (PEs). This has led to the broad adoption of tensor processing units (TPUs) and similarly dataflow-driven pipelines with low global data transfer and high clock frequencies [1].

Conventional TPUs reduce energy consumption and increase performance by reusing the values fetched from memory and registers [2] and thus, also reduce irregular intermediate memory accesses. However, the unceasing scaling of

deep learning models continues to increase memory access requirements, which are burdened by far more overhead than arithmetic. Many discriminative neural networks are tasked with learning the probability distribution associated with a dataset, and are thus tolerant to many types of errors. The soft-computing nature of deep learning means that many network architectures can handle errors, approximations, and even learn around them, but this error tolerance remains underutilized in modern accelerators [3], [4]. Eliminating the need to read and write redundant data, such as insignificant bits from weights that lack a meaningful representation of the trained task, provides an opportunity to keep pushing hardware along the same trajectory of neural network model growth [5], [6].

Here, we propose approximate processing elements (APEs) that replace direct quantization of inputs and weights for low-precision PEs in TPUs, and further reduce the overhead from each element-wise operation by developing pre-approximate units (PAUs) and sharing them between the APEs. By substantially reducing the critical path delay of a single PE that is typically required in regular, vectorized multiplication, the total processing time of a single forward-pass of data is significantly reduced in large-scale multi-element arrays. We assess the tolerance of a variety of neural networks and datasets to our approach to arithmetic approximation, and show negligible classification accuracy degradation, and even an improvement in several instances. We propose an approach to tiling APEs, and present the approximate tensor processing (APTPU), in which, we use the dynamic range unbiased multiplier (DRUM) [7] as a representative logarithmic multiplier and a lower-part OR adder (LOA) [8] to handle the accumulation operation in the multiply and accumulation (MAC) units. In doing so, we achieve a fair balance between accuracy, area, and power consumption. Our APTPU demonstrates up to $5.2\times$ and $4.4\times$ improvements in terms of TOPS/mm^2 and TOPS/W compared to a conventional TPU implemented in-house, while obtaining comparable accuracy.

The remainder of the paper is organized as follows. Section II provides the required background of the proposed work including a brief description of TPU architecture and prior approximate logic circuits. Section III describes the building blocks of the proposed APTPU architecture. Comprehensive simulation and synthesis results, as well as comparisons with previous works, are provided in Section IV. Finally,

Manuscript received 8 April 2022; revised 10 July 2022 and 23 August 2022; accepted 6 September 2022. Date of publication 23 September 2022; date of current version 9 December 2022. This article was recommended by Associate Editor T. Kawahara. (Corresponding author: Mohammed E. Elbity.)

Mohammed E. Elbity, Peyton S. Chandarana, Brendan Reidy, and Ramtin Zand are with the Department of Computer Science and Computer Engineering, University of South Carolina, Columbia, SC 29201 USA (e-mail: elbity@ieee.org).

Jason K. Eshraghian is with the Department of Electrical and Computer Engineering, University of California at Santa Cruz, Santa Cruz, CA 95064 USA.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSI.2022.3206262>.

Digital Object Identifier 10.1109/TCSI.2022.3206262

1549-8328 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

section V concludes the paper and discusses potential future work.

II. BACKGROUND

A. Tensor Processing Unit

The TPU architecture includes an array of PEs consisting of MAC units which implement matrix-matrix, vector-vector and matrix-vector multiplications. By using the systolic array, the TPU aims to reduce energy consumption and increase performance through reusing the values fetched from memory and registers [2] and thus, reducing the reads and writes from/to the buffers. Input data is fed in parallel to the array, and typically propagates in a diagonal wavefront [1], [9]. The microarchitecture of the MAC unit determines the data flow throughout the systolic array, and each data flow instruction influences power consumption, hardware utilization, and performance.

The dataflow pipeline is optimized to process a large number of MACs in parallel, which is the dominant operation in the forward pass of a neural network. A variety of data flow algorithms have been proposed, and can be broadly classed as input stationary data flow (IS) [10], weight stationary (WS) [1], output stationary (OS) [11], row stationary (RS) [12], and no-local reuse (NLR) [13], [14]. In the WS regime, each weight is pre-loaded onto a MAC unit. At each cycle, the input elements to be multiplied by the pinned weights are broadcasted among the MAC units within the array, producing partial sums every clock cycle. This process is vertically distributed over columns to generate the results from the bottom of the systolic array [1]. An almost identical process takes place for IS data flow, where the inputs are the fixed matrix and the weights are distributed horizontally. For OS data flow, outputs are pinned to the MAC units while the inputs and weights are propagated among the MAC units. For further details about the TPU's dataflow architecture, readers can refer to [15].

B. Approximate MAC Units

Several approximate MAC units have been proposed as alternatives for accurate multipliers and adders to leverage their power and area merits [7], [16], [17], [18], [19], [20], [21], [22], [23] and their potential in deep learning acceleration has been well-explored [24], [25], [26]. MAC units consist of two arithmetic stages, each of which may be approximated independently: multiplication, followed by accumulation with prior products.

1) *Approximate Multipliers*: Most approximate multipliers, including logarithmic multipliers, consist of two main components: low-precision arithmetic logic and a pre-processing unit which functions as the steering logic that prepares the operands for low-precision computation [7], [27]. Notable approximate multipliers trade-off between accuracy and power efficiency. For instance, the logarithmic multiplier proposed in [28] prioritizes accuracy, while the multipliers in [29] and [27] are optimized for power and delay efficiency.

Drawing inspiration from approximate image compression accelerators, approximate Booth multipliers have been used for

face detection in a convolutional neural network (CNN) accelerator enabling a high throughput [30], [31]. While acceptable performance was found for a set of computer vision tasks, Booth multipliers are likely to struggle in architectures that propagate and accumulate error signals through many layers, as is the case with recurrent and sequential models. This issue arises because Booth multipliers do not have symmetric error distributions centered around zero. While this may be tolerable for signal filtering and shallow networks, modern deep learning architectures may not be so tolerant to error propagation. In our APTPU design, without loss of generality, we employ the dynamic range unbiased multiplier (DRUM) [7] as a representative approximate multiplier. Not only is DRUM a well-known design that achieves a fair balance between accuracy, area, and power consumption, it also remains unbiased across multiple computational steps. When applied to systolic arrays where processing elements are reused, the error can be averaged out and reduced over multiple cycles which leads to better performance of more complex deep learning models as shown in Section IV.A. Nonetheless, other logarithmic multipliers can also be leveraged in the APTPU architecture by separating their steering logic and arithmetic logic parts into two separate blocks, as is explained in the next section.

2) *Approximate Adders*: Following multiplication, the resulting products must be accumulated with prior products to implement a MAC operation. The accumulation step can also be approximated using approximate adders. Various approximate adder designs have been widely proposed in the literature accomplishing power, area, and performance improvements [8], [32], [33], [34], [35], [36], [37]. Most approximate adders utilize the fact that the extended carry propagation rarely occurs, and thus adders can be divided into independent sub-adders to speed up the critical path. Moreover, to maintain the computation accuracy, the approximation occurs on the least significant bits of the operands, while keeping the most significant bits accurate. Here, we use a lower-part OR adder (LOA) [8] to handle the accumulation operation in the MAC units, which has been proven to yield acceptable performance in image detection [30]. This will be described in greater detail in the next section and assessed on a broader range of networks and tasks in Section IV.

III. PROPOSED APTPU ARCHITECTURE

The architecture of our proposed APTPU is classified as an OS systolic array, in which both the input feature map (IFMap) and weights are streamed into the APEs. This allows concurrent approximation of both the input and weight values within the APTPU architecture. In contrast, in the WS/IS data flow architectures, only the inputs or weights are streamed in the systolic array. Figure 1 demonstrates the overall architecture of the proposed APTPU which consists of five main components: weight/IFMap memory, FIFOs, a controller, pre-approximate unit (PAUs), and approximate processing elements (APEs).

The neural network model's weights and inputs are stored in the weight memory and IFMap memory, respectively. The controller manages accesses performed on the memory and the process of transferring the weights and IFMaps to the FIFOs

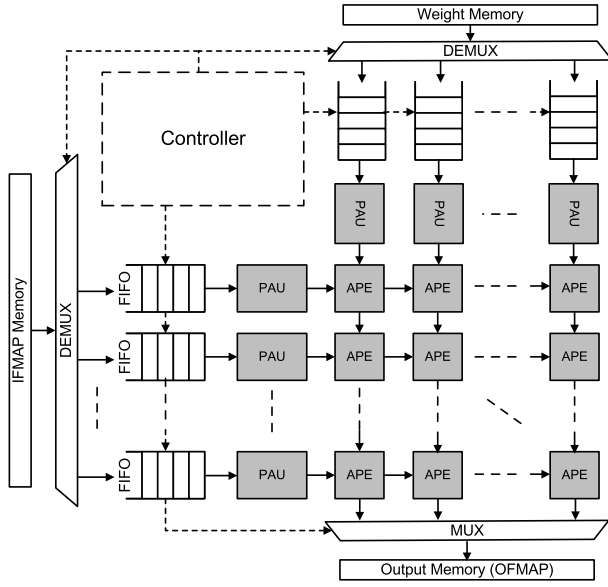


Fig. 1. The APTPU architecture including five main components: weight/IFMAP memories, FIFOs, controller, pre-approximate units (PAUs), and approximate processing elements (APEs).

according to the OS data flow algorithm. In the proposed APTPU architecture, we develop PAUs to handle the steering logic in approximate multipliers, in which operands with high precision (e.g., 32 bit) are dynamically truncated to low precision (e.g., 4 bit) ones and transferred to APEs that include a low precision multiplier, a Barrel shifter, and an approximate adder to realize the MAC operations. In most of the conventional approximate systolic array architectures [5], [38], [39], the exact MAC units in PEs are simply replaced by approximate MAC units. However, this leads to an increased critical path in PEs, and therefore, lower operating clock frequency and consequently a reduction in the total performance as each of the PEs in the systolic array is designed to perform the MAC operation in one clock cycle.

The main novelty of the proposed APTPU architecture is separating the steering logic of approximate multipliers and handling it outside the PEs in a separate PAU block, which reduces the critical path of PEs. Moreover, since the PAUs are responsible for truncating the operands, they could be placed between FIFOs and APEs on the path of the data stream flow to convert the high-precision operands, received from FIFO, to the low-precision ones transferred to the APEs. Thus, instead of using one PAU for each APE, we can share them among the PEs across different rows and columns of the systolic array. This approach adds an extra step (clock) compared to the conventional approximate systolic arrays, however, the reduction in the critical path delay is so significant that it results in overall performance improvements as is exhibited in Section IV. The details of our proposed PAU and APE architectures are described in the following.

A. Pre-Approximate Unit (PAU)

Figure 2 shows the architecture of the PAU. In the following, we describe the functionality of PAU's building blocks using a numerical example, i.e., IFMap=568 and Weight=-247,

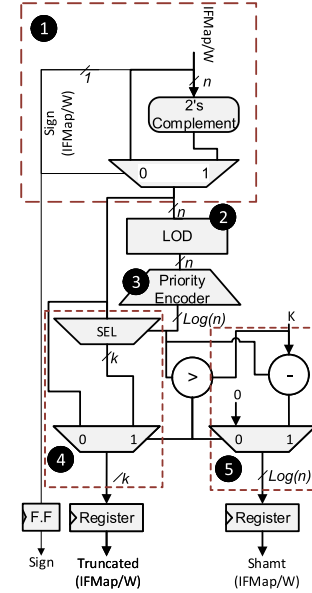


Fig. 2. Pre-approximate unit (PAU) schematic.

assuming that the low-precision multiplier resolution (k) and IFMap/ W bitwidths (n) are 4 and 16, respectively.

- 1) The signed to unsigned (S2U) unit generates the unsigned versions of the IFMap and weight along with its sign.

S2U Unit 1		
	IFMAP	Weight
In	0000001000111000 (568)	1111111100001001 (-247)
Out	0000001000111000 (568)	0000000011110111 (247)
Sign	0	1

- 2) The lead one detector (LOD) receives the S2U unsigned output and locates its leading HIGH bit.

LOD Unit 2		
	IFMAP	Weight
In	0000001000111000 (568)	0000000011110111 (247)
Out	0000001000000000 (512)	0000000010000000 (128)

- 3) The priority encoder receives the LOD output and generates the $\log_2()$ of it which represents the index of the lead HIGH bit (t) of the IFMap/weight. This value is used to control the selector (SEL) as well as calculating the shift amount ($shamt_{IFMap/W}$).

Priority Encoder 3		
	IFMAP	Weight
In (LOD Out)	0000001000000000 (512)	0000000010000000 (128)
Out (index t)	1001 ($\log_2(512) = 9$)	0111 ($\log_2(128) = 7$)

- 4) Next, the unsigned IFMap/ W is truncated starting from the bit at index t to index $t - k + 2$. The value at index $t - k + 1$ is set to "1" to compensate for the error caused by removing the bits from index $t - k$ to 0, where k is the pre-defined resolution of the low precision multiplier in the APE architecture. If $t > k$, then the selector selects the truncated unsigned IFMap/ W and feeds it to the APE. Otherwise, no truncation occurs

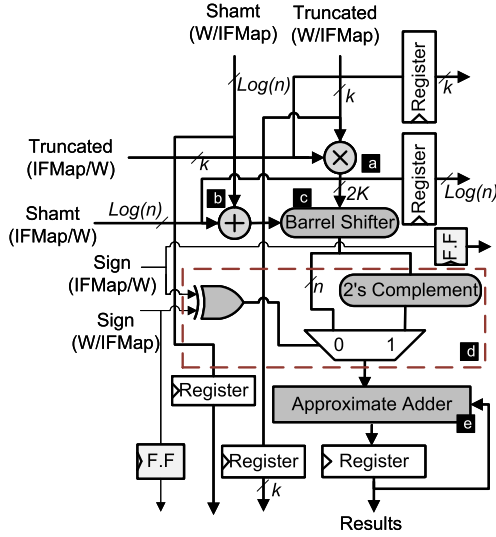


Fig. 3. Approximate processing element (APE) architecture.

and the unsigned $IFMap/W$ is transferred directly to the APE. This occurs when the operand is sufficiently small to fit within the resolution of the low precision multiplier in the APE, which mitigates the unnecessary loss of accuracy due to the approximation step.

Bit Selector Block 4		
	IFMAP	Weight
In	0000001000111000 (568)	0000000011110111 (247)
Truncated	1001 (9)	1111 (15)

- 5) Finally, if $t > k$, then $Shamt_{IFMap/W}$ is equivalent to $t - k + 1$, otherwise, there are no shifts required and $Shamt_{IFMap/W} = \text{zero}$ is passed to the APE.

Shift Amount Calculator 5		
	IFMAP	Weight
In (t)	1001 (9)	0111 (7)
Out ($shamt = t - k + 1$)	0110 (6)	0100 (4)

As shown in Fig. 2, each PAU unit is equipped with registers to keep its critical path as small as the APE, thus enabling an increased clock frequency.

B. Approximate Processing Element (APE)

The proposed APE receives the truncated bits, shift amount, and the sign of each operand from the corresponding PAUs, and then performs the MAC operation. The architecture of the APE is shown in Fig. 3 and described as follows:

- 1) The truncated bits $trunc_{IFMap/W}$ are multiplied using the low precision k -bit multiplier and the *product* is sent to the barrel shifter.

Low Precision k -bit Multiplier a			
	IFMAP	Weight	Out
Truncated Bits	1001 (9)	1111 (15)	10000111 (135)

- 2) The shift amount $Shamt_{IFMap/W}$ received from the PAUs are added to generate the $total_{shamt}$ amount.

Adder b			
	IFMAP	Weight	Out
$Shamt$	0110 (6)	0100 (4)	1010 (10)

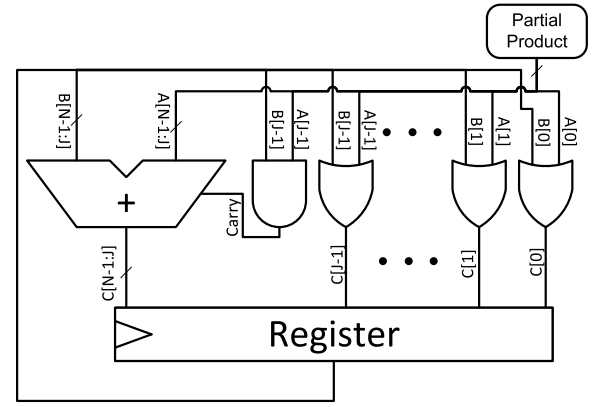


Fig. 4. LOA-based accumulator architecture.

- 3) The barrel shifter shifts the low precision multiplier result to the left by the $total_{shamt}$ value.

Barrel Shifter c		
In	Output of k -bit multiplier a	$total_{shamt}$
	10000111 (135)	1010 (10)
Out	100001110000000000 (138,240)	

- 4) The unsigned output of the barrel shifter is converted to a signed value using the XOR of the $sign_{IFMap/W}$ bits fed from the PAUs.

Unsigned to Sign Block d			
	$sign_{IFMap}$	$sign_W$	XOR
	0	1	1
In (output of c)	0000000000000000100001110000000000 (138,240)		
Out (2's compl.)	11111111111111011110010000000000 (-138,240)		

- 5) A lower-part OR adder (LOA) **e** is used to build the accumulator of the MAC unit as shown in Figure 4. The LOA consists of an array of OR gates with no carry propagation for the J least significant bits, i.e., $A[J-1:0]$ and $B[J-1:0]$, as well as an accurate adder that adds the most significant $N - J$ bits of the operands, i.e., $A[N-1:J]$ and $B[N-1:J]$. To increase the computational accuracy of this adder, an AND gate is used to generate the input *carry* for the accurate adder. The key advantage of the LOA is that the error is restricted to the least significant bits of each operand. The total bitwidth of the LOA accumulator (N) depends on the size of the systolic array (S) and IFMAP/W bitwidth (n) as expressed below,

$$N = 2n + \log_2(S) \quad (1)$$

In our design, we use 25% of the total bitwidth of LOA as the bitwidth of the approximate part, i.e., $J = N/4$.

C. APTPU Operation and Timing

The first step in APTPU operation is to store the IFMAPs and weights into the FIFOs, which is handled by the controller through DEMUX circuits. A $Write_{enable}$ signal is asserted to the input port of each FIFO and the data is transferred from memory to the FIFOs according to the OS dataflow. The depth of each FIFO in an APTPU architecture with a systolic array

Algorithm 1: APTPU Approximate MAC Unit

Input: a: Operand A, b: Operand B, k: low precision multiplier resolution

Output: ApproxProduct

```

1 def ApproxMultiply(a,b,k):
    /* PAU Start */
    /* Step ❶: Find the signs of the operands and convert them from signed to unsigned */
    2 signa = sign(a), signb = sign(b)
    3 ua = abs(a), ub = abs(b)
    /* Steps ❷ and ❸: Find the lead one bit of the operands. */
    4 ta = ⌊log2(ua)⌋
    5 tb = ⌊log2(ub)⌋
    /* Steps ❹ and ❺: Get the truncated operands and corresponding shift amounts. */
    6 trunca, shiftta = BitSelector(ua, ta, k)
    7 truncb, shifttb = BitSelector(ub, tb, k)
    8 trunca = trunca >> shiftta
    9 truncb = truncb >> shifttb
    /* Unbias */
    10 if trunca even then
    11     | trunca += 1
    12 if truncb even then
    13     | truncb += 1
    /* APE Start */
    /* Step ❶a: low precision multiplication */
    14 product = trunca * truncb
    /* Step ❶b: */
    15 totalshift = shiftta + shifttb
    /* Step ❶c: */
    16 ApproxProduct = product << totalshift
    /* Step ❶d: */
    17 if xor(signa, signb) then
    18     | ApproxProduct = -1 * ApproxProduct
    19 return ApproxProduct
    /* Dynamic Bit Selector (❹ and ❺) */
20 def BitSelector(uint, index, k):
    21 trunc = uint
    22 shift = 0
    23 if index > (k - 1) then
    24     mask = 0
    25     i = index - k
    26     while i ≤ lead do
    27         | mask += 2i
    28         | i += 1
    29     trunc = x & mask
    30     shift = index - (k - 1)
    31 return trunc, shift

```

size (S) of $M \times M$ is defined as,

$$FIFO_{Depth} = 2M - 1 \quad (2)$$

The number of clock cycles required to write the input and weight data into the FIFOs using two DEMUX circuits working in parallel can be calculated based on the equation below:

$$FIFO_{Cycles} = FIFO_{Depth} \times M = 2M^2 - M \quad (3)$$

After storing the input and weight data to all FIFOs, the controller asserts the *Read_{enable}* signal to each FIFO, and a new IFMAP/weight data is read in each clock cycle and transferred to the PAU blocks. The PAUs complete their task in one clock cycle and transfer data to the APEs. Typically, in an $M \times M$ output stationary systolic array, it takes M clock cycles to calculate the first element in the output matrix, and $3M - 2$ clock cycles to compute the last element [15]. However, since the proposed APTPU has an additional step for the PAU operation, the first element of the output matrix gets ready after $M + 1$ clock cycles, while the last element is ready after $3M - 1$ cycles. For example, in a $S = 8 \times 8$ systolic array, it takes 9 and 23 cycles to compute the first and last element in the output matrix, respectively.

D. Proposed Python Implementation for APTPU

To enable running machine learning (ML) applications on the proposed APTPU architecture without requiring an actual hardware implementation, we developed a Python program that implements the APTPU's functionality and implements the approximate multipliers as described in Algorithm 1. Similar to the APTPU architecture, the algorithm is divided into a PAU function followed by an APE function. In step 1, equivalent to ❶ in the PAU architecture, the signed to unsigned conversion happens through an absolute (*abs*) function. To find the index of the lead one bit (*t*) in the operands, the blocks ❷ and ❸ of the PAU are implemented by applying the floor function ($\lfloor x \rfloor$) to $\log_2$ of the operands. Next, a *BitSelector* function is created that receives the unsigned operands, index *t*, and resolution of the low precision multiplier (*k*) as inputs and returns the truncated operands and total shift amounts corresponding to steps ❹ and ❺ of the PAU, respectively. The APE function in the python program uses the truncated operands and shift amounts obtained by the PAU function to calculate the approximate output as shown in lines 14 to 19 of the algorithm.

IV. SIMULATION RESULTS AND DISCUSSIONS

To assess the performance and accuracy of our APTPU, we used a variety of deep learning models and datasets. We benchmarked accuracy, power, area, timing, tera-operations per-second (TOPS), TOPS per-watt (TOPS/W) and TOPS per-mm² (TOPS/mm²) across the parameter space of 8, 16 and 32 bits for IFMap/W precision (*n*) with the resolution of the low-precision multiplier (*k*) within the APE spanning from 3 to 6 bits, as described in the following subsections.

TABLE I
NORMALIZED MEAN ERROR DISTANCE ($NMED$) OF THE
APTPTU OVER 1,000 MATRIX MULTIPLICATIONS

n	k	Array Size (S)			
		8×8	16×16	32×32	64×64
8	3	5.511	37.997	25.971	56.054
	4	2.923	19.525	13.193	28.687
	5	1.535	10.835	7.000	14.970
	6	0.896	6.569	4.121	8.866
16	3	26.633	9.631	36.262	34.928
	4	11.889	4.826	18.444	17.471
	5	5.979	2.422	9.223	8.716
	6	2.853	1.210	4.562	4.356
32	3	10.245	52.815	323.318	691.444
	4	8.287	48.279	308.108	673.453
	5	7.293	45.772	300.252	662.464
	6	6.799	44.539	295.793	656.523

A. Error Analysis and Error-Tolerant Applications

Herein, we provide a system-level error analysis for the APTPTU architecture and investigate its accuracy for some error-tolerant applications to demonstrate its applicability in practical scenarios.

1) *Error Analysis*: To measure the error introduced by the APTPTU architecture at the system level, we simulated matrix multiplications using the developed Python program for the APTPTU with varying sizes of arrays (S), integers bit widths (n), and low-precision multiplier resolution (k). We performed 1,000 randomly generated matrix multiplications for each of the $S = M \times M$, n , and k combinations. To compute the error resulting from the use of APTPTU compared to that of the exact TPU, we utilize a normalized version of the average Mean Error Distance (MED_{avg}) introduced in [5] as expressed in Equation 4,

$$NMED = \frac{MED_{avg}}{|Avg(P_{exact})|} = \frac{\sum [|P_{exact}(i, j) - P_{approx}(i, j)|]}{M \times M \times |Avg(P_{exact})|} \quad (4)$$

where $NMED$ is the normalized MED_{avg} , and P_{exact} and P_{approx} are the products of matrix multiplication using non-approximate TPU, and APTPTU, respectively. This metric allows the error to be analyzed with respect to the ratio between the MED_{avg} and the average value inside the matrices to allow for the error metrics to be comparable between different values of S and n . Table I lists the $NMED$ values obtained for different combinations of S , n , and k . As seen in the table, as the resolution of low-precision multiplier (k) increases, the error decreases in all matrix sizes with the minimal error being accomplished at $k = 6$. We can also see that, in general, as the size of the matrix and input bitwidth (n) increase the error does as well.

2) *Error-Tolerant Application*: To investigate the applicability of the proposed APTPTU in practical applications, we utilize the proposed design for a sentiment analysis application using IMDB movie review dataset, as well as three different image classification tasks including the MNIST handwritten digits [40], EMNIST handwritten letters [41], and Fashion MNIST [42]. The three image datasets were trained on LeNet-5 [40] CNN architecture. For IMDB dataset, we used an MLP-5 dense network with five fully-connected layers.

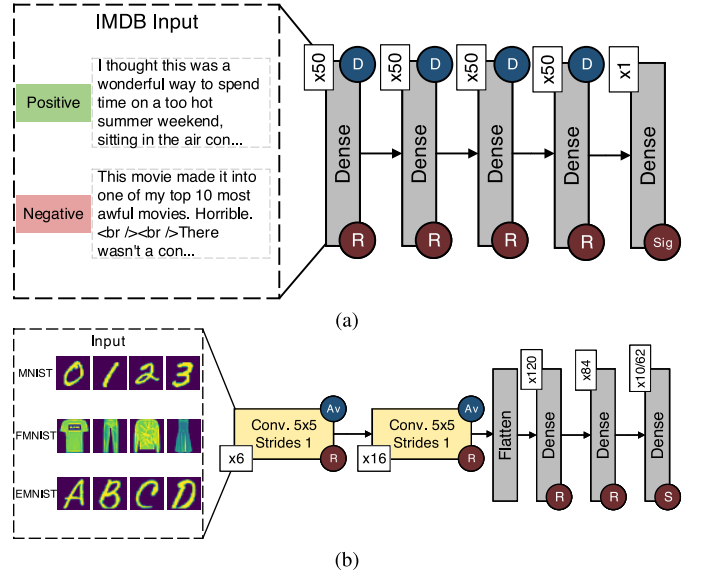


Fig. 5. Model architectures for (a) MLP-5 for IMDB classification and (b) LeNet-5 for MNIST, FMNIST, and EMNIST classification. Av AveragePooling2D, D Dropout, Sig Sigmoid, R ReLU, S Softmax.

TABLE II
APTPTU VS. TPU ACCURACY - BIT PRECISION COMPARISON

n	k	Accuracy (%)			
		IMDB	MNIST	FMNIST	EMNIST
8	3	85.2	63.5	39.0	13.6
	4	85.2	62.8	42.4	14.0
	5	85.6	62.5	41.5	13.2
	6	85.6	63.4	41.4	12.9
	Normal	85.6	63.1	41.0	13.4
16	3	86.3	92.9	67.5	36.2
	4	86.2	91.8	70.7	34.0
	5	86.6	90.7	69.3	32.7
	6	86.3	91.0	69.3	30.4
	Normal	86.4	90.7	69.6	31.2
32	3	86.2	98.8	74.3	76.7
	4	86.4	98.6	75.9	75.2
	5	86.3	98.7	75.8	75.4
	6	86.4	98.5	75.7	74.0
	Normal	86.4	98.6	75.8	74.4

A detailed description of both architectures is provided in Figure 5. We chose smaller networks to reduce the runtime of our experiments. Performing accuracy evaluations on larger models is particularly time-consuming as we had to override the base TensorFlow modules to replace multiplication steps to approximate arithmetic blocks (Algorithm 1).

Each network is first trained using full precision floating point values and then the inputs, weights and activations are quantized. Here, we have not used a quantization-aware training approach in order to measure the worst-case accuracy, which assumes a pre-trained model is applied directly to our APTPTU. A non-approximate TPU with 8, 16, and 32 bits for IFMap/W is used here as a baseline. The validation accuracy results across all cases are provided in Table II. The non-approximate TPU baseline is labeled 'Normal'. The average accuracy change from baseline to APTPTU across datasets and bit-lengths for multiplier precision of $k = 3$ is

TABLE III
THE ACCURACY OBTAINED BY BASELINE TPU WITH LOW
PRECISION MAC UNITS WITHOUT APPROXIMATION

k	Accuracy (%)			
	IMDB	MNIST	FMNIST	EMNIST
3	81.4	40.5	22.2	6.4
4	83.4	42.4	41.3	7.2
5	85.0	51.7	34.1	10.9
6	85.5	58.8	42.3	12.5

+0.49%, $k = 4$ is +0.85%, $k = 5$ is +0.42%, and $k = 6$ is -0.17%, where k is the resolution of low-precision multiplier in APE. In general, the accuracy of the APTPU matches or outperforms the results of the non-approximate simulation for the former three values of multiplier precision. We expect this is due to the quantization and approximation noise acting as a regularizer to the neural network which can reduce overfitting [43], [44], [45]. For the more complex tasks, such as FMNIST and EMNIST classification, a lower bit-width degrades accuracy evenly across the high-precision baseline and the APTPU.

To verify that the high accuracy of the APTPU is a result of the approximation techniques that have been applied, and similar accuracy values cannot be achieved by reducing the precision of MAC units, we evaluated the accuracy of baseline TPU with IFMap/W bits (n) from 3 to 6 bits without approximation. The low accuracy results of the low-precision baseline listed in Table III confirms that the high accuracy of the APTPU is a result of the approximation techniques. However, the accuracy of low-precision TPU can be potentially improved by training using quantization-aware techniques [4].

B. Area and Power Consumption

Here, we investigate the area occupation and power consumption of the APTPU using Synopsys Design Compiler along with the 45nm Nangate Open Cell Library. The timing constraints in all experiments are set to a clock period of 10ns with an uncertainty of 2%, and a clock network delay of 1ns. Our developed parameterizable RTL enabled the experiments to span across values $k = 3, 4, 5, 6$ for the low-precision multiplier resolution with IFMap/W bitwidth (n) spanning between 8, 16, and 32, for differently sized systolic arrays $S = 8 \times 8, 16 \times 16, 32 \times 32$, and 64×64 .

As the processing element is the most important module in the systolic array architecture, we measure the power consumption and area of our proposed APE compared to the baseline TPU's PE and DRUM-based PE, in which a DRUM approximate multiplier [7] replaces the multiplier in baseline PEs. Sweeping across various values of input feature map and weight (IFMap/W) precision (n) and the low-precision multiplier resolution (k) gives an accurate estimate for the possible design options available. Table IV exhibits the area and power consumption for each PE. The synthesis reports show that the proposed APE achieves up to 75.6% and 49.2% area reduction compared to baseline PE and DRUM-based PE, respectively, and up to 82.8% and 32.5% decrease in

TABLE IV
AREA AND POWER CONSUMPTION COMPARISON OF PEs

n	k	Power Consumption (mW)			Area (μm^2)		
		TPU PE	DRUM PE	APTPU APE	TPU PE	DRUM PE	APTPU APE
8	3		0.0497	0.0390		757	561
	4	0.117	0.0553	0.0445	1695	824	632
	5		0.0603	0.0502		867	706
	6		0.0646	0.0562		947	791
16	3		0.0850	0.0590		1552	889
	4	0.23	0.0934	0.0661	3093	1664	975
	5		0.1041	0.0741		1781	1074
	6		0.1171	0.0821		1933	1160
32	3		0.1543	0.1042		3347	1759
	4	0.605	0.1662	0.1132	7596	3648	1853
	5		0.1773	0.1222		3692	1954
	6		0.1892	0.1317		3776	2096

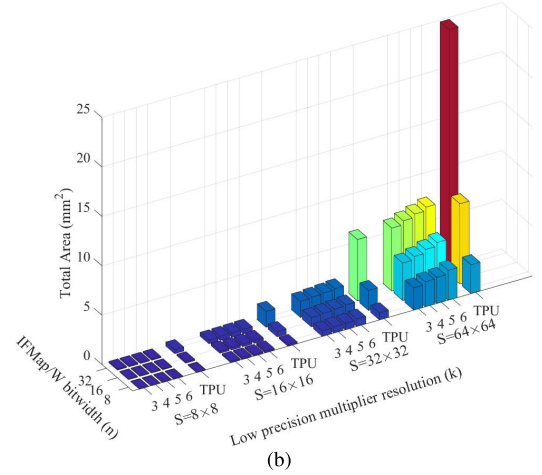
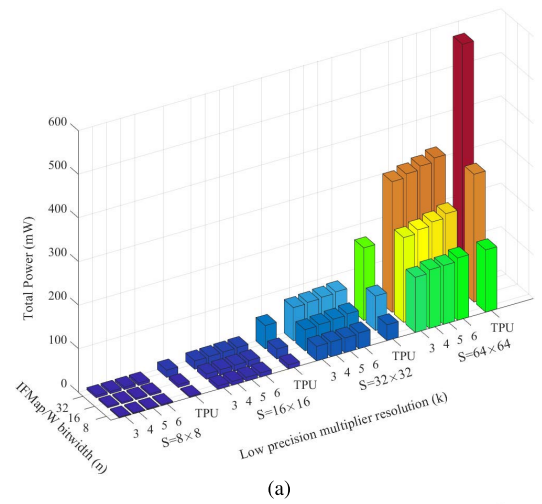


Fig. 6. Hardware implementation results. (a) Power consumption, (b) Area.

power consumption compared to TPU PE and DRUM-based PE, respectively.

Table V and Figure 6 show the power consumption and area occupation of the APTPU compared to the baseline TPU, where an average reduction of 41.3% and 55.3% are achieved, respectively, across all experiments. This can be attributed to two features: 1) the use of low-precision multipliers, and 2) the steering logic has been moved outside the matrix-multiply unit to enable sharing between processing elements.

TABLE V
AREA AND POWER CONSUMPTION RESULTS

S	n	k	Area (mm^2)		Total Power (mW)	
			APTPU	TPU	APTPU	TPU
8×8	8	3	0.0345	0.0434	2.04	2.49
		4	0.0392		2.24	
		5	0.0414		2.35	
	6	6	0.0475	0.1287	2.53	6.42
		3	0.0573		3.09	
		4	0.0630		3.32	
	5	5	0.0695	0.3993	3.64	16.61
		6	0.0753		3.94	
		3	0.1001		4.93	
	4	4	0.1078	1.6025	5.20	53.59
		5	0.1131		5.45	
		6	0.1193		5.75	
16×16	8	3	0.1371	0.1788	7.94	9.54
		4	0.1557		8.59	
		5	0.1642		8.83	
	6	6	0.1888	0.5215	9.50	22.70
		3	0.2245		12.08	
		4	0.2467		12.83	
	5	5	0.2713	1.6025	13.75	53.59
		6	0.2943		14.85	
		3	0.3845		19.04	
	4	4	0.4127	2.0152	19.89	79.01
		5	0.4350		20.71	
		6	0.4590		21.66	
32×32	8	3	0.5534	0.7134	33.28	36.14
		4	0.6226		34.02	
		5	0.6497		33.87	
	6	6	0.7458	2.0152	36.15	79.01
		3	0.9433		48.57	
		4	1.0263		51.00	
	5	5	1.1162	6.2064	53.81	166.45
		6	1.2123		56.80	
		3	1.6067		76.09	
	4	4	1.7102	2.9121	78.68	142.60
		5	1.7966		81.11	
		6	1.8891		84.15	
64×64	8	3	2.2407	2.9121	126.40	142.60
		4	2.5199		134.19	
		5	2.6329		134.61	
	6	6	3.0134	8.1257	142.80	293.10
		3	3.7919		194.50	
		4	4.1210		202.76	
	5	5	4.5059	24.8041	211.67	568.22
		6	4.8613		221.26	
		3	6.3683		299.59	
	4	4	6.7712	24.8041	308.14	568.22
		5	7.1203		316.83	
		6	7.4875		326.09	

Both of these contribute to the reduction of size of each APE as also reported in Table IV.

C. Timing

The timing analysis is also performed in Synopsys Design Compiler. We use the non-approximate TPU as a baseline, in addition to a DRUM-based TPU, in which exact multipliers in the processing elements are simply replaced with a DRUM approximate multiplier, without leveraging shared PAUs to implement the steering logic. This allows us to separate the performance benefits that are only a result of using approximate units, from those that are a result of design reuse, *i.e.* sharing steering logic among APEs proposed in the APTPU. Table VI provides the performance comparison in terms of giga operation per second (GOPS). The APTPU outperforms

TABLE VI
PERFORMANCE COMPARISON BETWEEN APTPU, DRUM TPU, AND BASELINE TPU FOR VARIOUS SYSTOLIC ARRAY SIZES IN TERMS OF GOPS

Systolic Array Size (S)	8×8	16×16	32×32	64×64
TPU Systolic	16.45	65.47	265.63	1050.26
DRUM Systolic	15.17	60.24	239.81	952.56
APTPU Systolic	24.49	92.52	351.41	1308.18

the non-approximate TPU and DRUM-TPU baselines under all conditions, while DRUM-TPU shows lower performance compared to baseline TPU. These results indicate that the timing improvement of our APTPU is not a direct result of approximate computing units, but rather, the use of shared PAUs proposed herein.

In addition, we employed SCALE-Sim [15], a cycle accurate simulator for machine learning accelerators, to obtain the required number of clock cycles for each layer in a variety of neural network architectures. As before, we used LeNet-5 and MLP5, but additionally obtained a cycle count on MobileNet [46] and YOLO-Tiny [47] to also assess performance on larger models for embedded deployment. The number of cycles is measured using the OS data flow regime of the APTPU and multiplied by the frequency obtained from the critical path timing measurements to find the total execution time, *i.e.*, the time taken from passing an input image to the network to obtain an output at the final layer. Figure 7 shows the total execution time for the aforementioned ML models for various S and n values. To exhibit the timing benefits of APTPU at scale, we extended our experiments to include a wide range of ML workloads and applications such as deep CNN models for face recognition, deep recurrent neural networks (RNNs) for speech recognition, transformers for language translations, and recommendation systems. The results are shown in Fig. 8, where the workloads are categorized into five classes based on the execution time. We use the worst-case timing of the low-precision multiplier for our APTPU and the best-case for the baseline TPU and DRUM TPU. Regardless, our proposed APTPU runs faster than both baselines for all test cases. In particular, the APTPU achieves an average of $1.4\times$ and $1.5\times$ speedup compared to baseline TPU and DRUM TPU, respectively, across all of the conducted experiments.

D. Performance

To provide a streamlined comparison that factors in timing, power consumption, and area, we calculated the TOPS/W and TOPS/ mm^2 . Table VII provides a performance comparison between the TPU, DRUM-TPU, and APTPU, for $S = 64 \times 64$ using different IFMap/W bitwidths (n), and multiplier precision resolution (k). For all cases, our APTPU outperforms the baseline TPU and DRUM TPU. In particular, the APTPU achieves $1.2\text{--}4.4\times$ and $1.4\text{--}2.4\times$ improvement in TOPS/W compared to the baseline TPU and DRUM TPU, respectively. In terms of TOPS/ mm^2 , the APTPU achieves $1.2\text{--}5.3\times$ and $1.8\text{--}2.7\times$ improvement compared to TPU and DRUM TPU, respectively. As network bit precision is varied, the TOPS/W metric of all TPUs scales at a constant rate over the measured

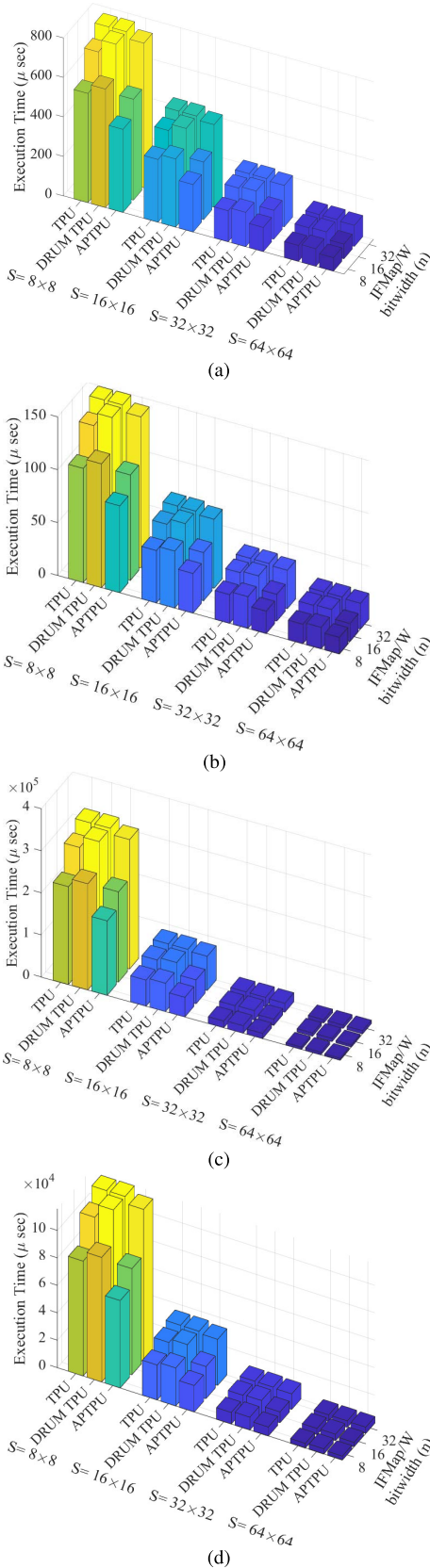


Fig. 7. Total execution time of various deep learning models on baseline TPU, DRUM TPU and APTPU. (a) MLP5, (b) LeNet-5, (c) YOLO-Tiny, (d) MobileNet.

range, while for TOPS/mm², the APTPU improves at a faster rate than both the TPU and DRUM TPU as IFMap/w bitwidth

TABLE VII
COMPARISON BETWEEN APTPU PERFORMANCE AND BASELINE TPU AND DRUM-BASED TPU FOR $S = 64 \times 64$

n	k	TOPS/W			TOPS/mm ²		
		TPU	DRUM TPU	APTPU	TPU	DRUM TPU	APTPU
8	3		6.86	10.35	0.36	0.31	0.58
	4	7.37	6.7	9.38		0.28	0.5
	5		6.61	9.86		0.27	0.5
	6		6.54	9.32		0.25	0.44
16	3		2.97	6.08	0.1	0.13	0.31
	4	2.69	2.94	5.72		0.12	0.28
	5		2.9	5.52		0.11	0.26
	6		2.86	5.24		0.11	0.24
32	3		1.5	3.03	0.03	0.06	0.14
	4	1.34	1.48	2.62		0.06	0.12
	5		1.48	2.85		0.06	0.13
	6		1.47	2.80		0.05	0.12

TABLE VIII
AREA AND POWER CONSUMPTION REDUCTION FOR THE ENTIRE APTPU COMPARED TO THE IN-HOUSE IMPLEMENTED TPU FOR VARIOUS SYSTOLIC ARRAY SIZES

Size		8 × 8	16 × 16	32 × 32	64 × 64
Area (mm ²)	TPU	0.5	1.98	8.02	32.16
	APTPU	0.2	0.78	3.24	13.06
	Reduction	2.5×	2.54×	2.48×	2.46×
Power (mW)	TPU	14.7	59.67	238.35	955.36
	APTPU	11.61	46.57	187.54	754.34
	Reduction	1.27×	1.28×	1.27×	1.27×

(n) is decreased. In general, as n is doubled, the TOPS/mm² of baseline TPU drops approximately by a factor of 4, while that of the APTPU only drops by a factor of 2. Moreover, as listed in Table VII, increasing the multiplier precision resolution (k) in approximate MAC units can lead to a slight decrease in both TOPS/W and TOPS/mm² of the DRUM TPU and APTPU designs.

E. System Analysis

As mentioned in Section III, the proposed APTPU aims to improve the systolic array component of the TPU by replacing the large and power-hungry accurate MAC unit with an approximate MAC unit. Although the other components including FIFOs and controller are not improved in our approach, the systolic array itself is a crucial component and it occupies between 77%-80% of the entire TPU's area and consumes approximately, 50%-89% of its power consumption depending on the systolic array size (S), MAC unit precision (n), and the size of the low-precision multiplier (k). Figure 9 shows the in-house implemented TPU chip layout and exhibits the placement of the systolic array component on the entire layout. The size of the systolic array in the in-house baseline TPU chip is $S = 8 \times 8$ and has been implemented using 65 nm technology at a clock frequency of 100 MHz. Moreover, we conducted additional experiments to assess the effect of the achieved systolic array improvements on the entire system. Table VIII lists the area and power consumption of our proposed APTPU, including the entire system components, compared to the baseline TPU. The results obtained show

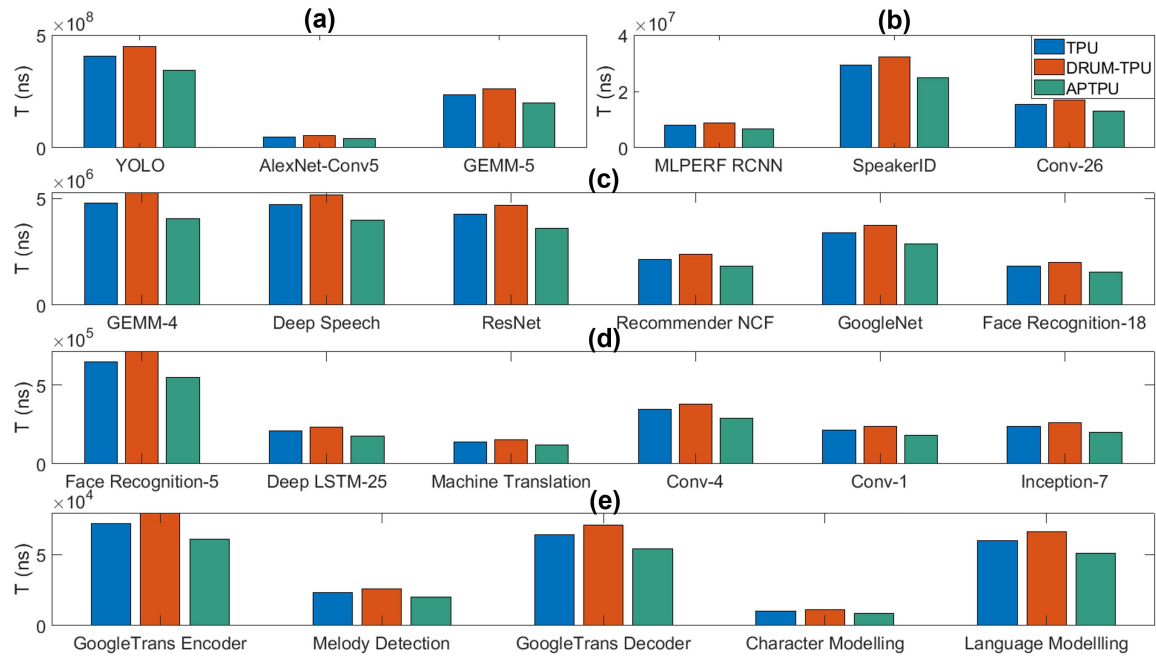


Fig. 8. Execution time (T) of various workload on TPU, DRUM TPU, and APTPU, assuming $S = 64 \times 64$ and $n = 8$. (a) Very large ($100ms < T < 1s$), (b) large ($10ms < T < 100ms$), (c) Medium ($1ms < T < 10ms$), (d) Small ($0.1ms < T < 1ms$), (e) Very small ($0.01ms < T < 0.1ms$).

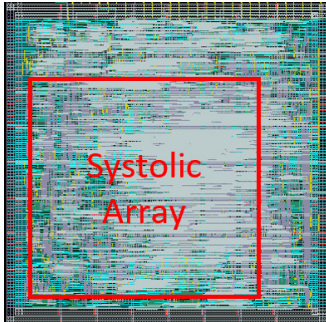


Fig. 9. In-house designed TPU chip layout.

that the APTPU reduces the overall chip area and power consumption by approximately $2.5\times$ and $1.2\times$, respectively.

F. Comparison to the State-of-the-Art

Here, we compare our APTPU with four other state-of-the-art approximate systolic arrays [5], [6], [38], [39]. For a fair comparison, we synthesize APTPU with the same design constraints and specifications existing in each of the previous designs in terms of IFMap/W precision (n), resolution of the low-precision multiplier (k), size of the systolic array (S), depth of the FIFO, and clock frequency. All the designs are synthesized using the 45nm technology.

Table IX shows a comparison between our proposed APTPU against three of the most efficient approximate systolic arrays in the literature. In this comparison, the design hyperparameters are set as $S = 32 \times 32$, $n = 8$, and $k = 5$, which are the values used in [39]. The approximate computing scheme in [39] is based on replacing the full adder in the Baugh-Wooley multiplier and the accumulator with an approximate full adder.

TABLE IX
COMPARISON BETWEEN APTPU AND OTHER APPROXIMATE SYSTOLIC ARRAYS WITH $S=32 \times 32$, $N=8$, AND $K=5$

Design	Power (mW)	Delay (ns)	PDP(pJ)	MED
[5]	251.96	1.83	461.08	69
[38]	268.74	2.53	679.91	57
[39]	258.64	2.51	649.18	78
This Work	206.1218	1.6	329.79	866

A similar approach has been adopted in [5] and [38], in which they proposed approximate partial product units (PPUs) and approximate adder and utilized them in the design of the PE of the systolic array. Using their proposed approximation approach, they could shrink the critical path of the PE with a few modifications in its microarchitecture at the cost of a static approximation error. On the other hand, the approximate scheme used in the APTPU is dynamic and the error is mostly affecting the least significant bits. Additionally, our APTPU utilizes shared pre-approximate units (PAUs) along with approximate adders which further reduce the critical path delay of the entire systolic array. As listed in Table IX, our APTPU achieves $1.14 \times -1.58 \times$ and $1.4 \times -2 \times$ reduction in the delay and power-delay-product (PDP) value, respectively, compared to the other approximate systolic arrays. It should be noted that, as listed in the last column of the table, the APTPU architecture introduces more Mean Error Distance (MED) compared to the previous designs for isolated matrix multiplications. However, as elaborated in Section IV.A, these errors are averaged out and reduced over multiple cycles and do not impact the performance of the APTPU while used for error-tolerant ML applications that are the targeted workloads for TPUs. Therefore, over-optimizing the design to gain better accuracy for matrix multiplication is not necessary.

TABLE X

COMPARISON BETWEEN APTPU AND SIMILAR APPROACH IN [6]
FOR $S=16 \times 16$, $N=32$, $K=8$, AND FIFO DEPTH = 64, WHILE
CONSTRAINING THE DESIGN TO WORK UNDER 250 MHz

Metric	Power (mW)	Area (mm ²)
QUORA [6]	367.8	2.6
This work	183.6	1.46

QUORA is another approximate systolic array architecture proposed in [6] that utilizes shared precision scaling units as well as approximate processing elements (APEs). Quora has four different precision scaling schemes namely, up/down precision scaling, precision scaling with error monitoring, precision scaling with error compensation, and dynamic precision scaling. The precision scaling approaches truncate some number of least significant bits from the operands, thus parts of the PEs can be disabled through power gating or clock gating mechanisms, which results in power savings. However, as opposed to our APTPU, their APE operates over multiple clock cycles and its functionality is determined by some control signals due to having different functional units. In contrast, the APE in our APTPU has a low precision multiplier and an approximate adder with no control overhead. As listed in Table X, while constraining the design to work with 250 MHz similar to [6] for a fair comparison, our APTPU achieves $2\times$ and $1.78\times$ reduction in power consumption and area, respectively, for a systolic array of size $S = 16 \times 16$, $n = 32$, $k = 8$, and FIFO depth = 64 design specifications used in [6].

V. CONCLUSION

We propose a novel approach to leveraging approximate arithmetic logic units within the TPU architecture. The shareable components of the approximate circuits are integrated into the pre-approximate units (PAUs), and re-used among the approximate processing elements (APEs) in the systolic array. This enables significant improvements compared to baseline TPU architectures, as well as TPU designs that simply replace MAC units with approximate units. Our simulations span across a broad design space while quantifying the trade-offs in using approximate logic for a variety of machine learning workloads. We perform a comprehensive analysis across neural network accuracy, timing, area, power, and performance. We show that our proposed APTPU outperforms conventional designs in terms of area, power, and timing while realizing comparable accuracy. In particular, the proposed APTPU's approximate systolic array achieves up to $5.2\times$ TOPS/W and $4.4\times$ TOPS/mm² improvement compared to that of a conventional TPU architecture, while the entire APTPU system achieves a reduction of $2.5\times$ and $1.2\times$ area and power consumption, respectively. Furthermore, simulation results and synthesis report obtained show that the approximation scheme in the proposed APTPU achieves a significant reduction in power, delay, and area compared to four of the most efficient approximate systolic arrays in the literature while using similar design specifications and synthesis constraints. Future work will focus on implementing a mixed-precision APTPU which supports a variety of data flow architectures.

REFERENCES

- [1] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
- [2] N. Jouppi, C. Young, N. Patil, and D. Patterson, "Motivation for and evaluation of the first tensor processing unit," *IEEE Micro*, vol. 38, no. 3, pp. 10–19, May/Jun. 2018.
- [3] X. Lin, C. Zhao, and W. Pan, "Towards accurate binary convolutional neural network," 2017, *arXiv:1711.11294*.
- [4] J. Lee, J. K. Eshraghian, S. Kim, K. Eshraghian, and K. Cho, "Quantized convolutional neural network implementation on a parallel-connected memristor crossbar array for edge AI platforms," *J. Nanosci. Nanotechnol.*, vol. 21, no. 3, pp. 1854–1861, Mar. 2021.
- [5] H. Waris, C. Wang, W. Liu, and F. Lombardi, "AxSA: On the design of high-performance and power-efficient approximate systolic arrays for matrix multiplication," *J. Signal Process. Syst.*, vol. 93, no. 6, pp. 605–615, Jun. 2021.
- [6] S. Venkataramani, V. K. Chippa, S. T. Chakradhar, K. Roy, and A. Raghunathan, "Quality programmable vector processors for approximate computing," in *Proc. 46th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Dec. 2013, pp. 1–12.
- [7] S. Hashemi, R. I. Bahar, and S. Reda, "DRUM: A dynamic range unbiased multiplier for approximate applications," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2015, pp. 418–425.
- [8] A. Dalloo, A. Najafi, and A. Garcia-Ortiz, "Systematic design of an approximate adder: The optimized lower part constant—Or adder," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 8, pp. 1595–1599, Aug. 2018.
- [9] J. Shen, H. Ren, Z. Zhang, J. Wu, W. Pan, and Z. Jiang, "A high-performance systolic array accelerator dedicated for CNN," in *Proc. IEEE 19th Int. Conf. Commun. Technol. (ICCT)*, Oct. 2019, pp. 1200–1204.
- [10] S. Lym and M. Erez, "FlexSA: Flexible systolic array architecture for efficient pruned DNN model training," 2020, *arXiv:2004.13027*.
- [11] S. Guo *et al.*, "A systolic SNN inference accelerator and its co-optimized software framework," in *Proc. Great Lakes Symp. VLSI*, May 2019, pp. 63–68.
- [12] Y. H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, May 2017.
- [13] E. Qin *et al.*, "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Feb. 2020, pp. 58–70.
- [14] H. Lim, V. Piuri, and E. E. Swartzlander, "A serial-parallel architecture for two-dimensional discrete cosine and inverse discrete cosine transforms," *IEEE Trans. Comput.*, vol. 49, no. 12, pp. 1297–1309, Dec. 2000.
- [15] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "SCALE-Sim: Systolic CNN accelerator simulator," 2018, *arXiv:1811.02883*.
- [16] J. N. Mitchell, "Computer multiplication and division using binary logarithms," *IRE Trans. Electron. Comput.*, vol. 11, no. 4, pp. 512–517, Aug. 1962.
- [17] B.-G. Nam, H. Kim, and H.-J. Yoo, "Power and area-efficient unified computation of vector and elementary functions for handheld 3D graphics systems," *IEEE Trans. Comput.*, vol. 57, no. 4, pp. 490–504, Apr. 2008.
- [18] M. B. Sullivan and E. E. Swartzlander, "Truncated error correction for flexible approximate multiplication," in *Proc. Conf. Rec. Forty 6th Asilomar Conf. Signals, Syst. Comput. (ASILOMAR)*, Nov. 2012, pp. 355–359.
- [19] K. Bhardwaj, P. S. Mane, and J. Henkel, "Power- and area-efficient approximate Wallace tree multiplier for error-resilient systems," in *Proc. 15th Int. Symp. Quality Electron. Design*, Mar. 2014, pp. 263–269.
- [20] B. Boro, K. M. Reddy, Y. B. N. Kumar, and M. H. Vasantha, "Approximate radix-8 booth multiplier for low power and high speed applications," *Microelectron. J.*, vol. 101, Jul. 2020, Art. no. 104816.
- [21] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, and F. Lombardi, "Design of approximate radix-4 booth multipliers for error-tolerant computing," *IEEE Trans. Comput.*, vol. 66, no. 8, pp. 1435–1441, Aug. 2017.
- [22] M. Amudha and K. Sivasubramanian, "Design of low-error fixed-width modified booth multiplier," *Int. J. Electron. Comput. Sci. Eng.*, vol. 1, no. 2, pp. 522–531, 2012.

- [23] J. P. Wang, S. R. Kuang, and S. C. Liang, "High-accuracy fixed-width modified booth multipliers for lossy applications," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 19, no. 1, pp. 52–60, Jan. 2011.
- [24] M. E. Elbity, H.-W. Son, D.-Y. Lee, and H. Kim, "High speed, approximate arithmetic based convolutional neural network accelerator," in *Proc. Int. SoC Design Conf. (ISOCC)*, Oct. 2020, pp. 71–72.
- [25] V. Kumar and R. Kant, "Approximate computing for machine learning," in *Proc. 2nd Int. Conf. Commun., Comput. Netw.*, 2019, pp. 607–613.
- [26] H. Younes, A. Ibrahim, M. Rizk, and M. Valle, "Algorithmic level approximate computing for machine learning classifiers," in *Proc. 26th IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Nov. 2019, pp. 113–114.
- [27] P. Yin, C. Wang, H. Waris, W. Liu, Y. Han, and F. Lombardi, "Design and analysis of energy-efficient dynamic range approximate logarithmic multipliers for machine learning," *IEEE Trans. Sustain. Comput.*, vol. 6, no. 4, pp. 612–625, Oct. 2021.
- [28] M. S. Ansari, B. F. Cockburn, and J. Han, "An improved logarithmic multiplier for energy-efficient neural computing," *IEEE Trans. Comput.*, vol. 70, no. 4, pp. 614–625, Apr. 2021.
- [29] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi, and F. Lombardi, "Design and evaluation of approximate logarithmic multipliers for low power error-tolerant applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 9, pp. 2856–2868, Sep. 2018.
- [30] H. Mo *et al.*, "A 1.17 TOPS/W, 150 fps accelerator for multi-face detection and alignment," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, pp. 1–6.
- [31] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proc. IEEE*, vol. 108, no. 12, pp. 2108–2135, Dec. 2020.
- [32] P. Balasubramanian and D. L. Maskell, "Hardware optimized and error reduced approximate adder," *Electronics*, vol. 8, no. 11, p. 1212, Oct. 2019.
- [33] R. Nayar, P. Balasubramanian, and D. L. Maskell, "Hardware optimized approximate adder with normal error distribution," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Jul. 2020, pp. 84–89.
- [34] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 1, pp. 124–137, Jan. 2013.
- [35] S. Mittal, "A survey of techniques for approximate computing," *ACM Comput. Surv.*, vol. 48, no. 4, pp. 1–33, 2016.
- [36] I. C. Lin, Y. M. Yang, and C. C. Lin, "High-performance low-power carry speculative addition with variable latency," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1591–1603, Sep. 2015.
- [37] P. L. Lahari, M. Bharathi, and Y. J. Shirur, "An efficient truncated MAC using approximate adders for image and video processing applications," in *Proc. 4th Int. Conf. Trends Electron. Informat. (ICOEI)*, Jun. 2020, pp. 1039–1043.
- [38] H. Waris, C. Wang, W. Liu, and F. Lombardi, "Design and evaluation of a power-efficient approximate systolic array architecture for matrix multiplication," in *Proc. IEEE Int. Workshop Signal Process. Syst. (SiPS)*, Oct. 2019, pp. 13–18.
- [39] K. Chen, F. Lombardi, and J. Han, "Matrix multiplication by an inexact systolic array," in *Proc. IEEE/ACM Int. Symp. Nanosc. Archit.*, Jul. 2015, pp. 151–156.
- [40] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [41] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 2921–2926.
- [42] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [43] X. Xu *et al.*, "Quantization of fully convolutional networks for accurate biomedical image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8300–8308.
- [44] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, "Incremental network quantization: Towards lossless CNNs with low-precision weights," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–14.
- [45] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neuro-computing*, vol. 461, pp. 370–403, Oct. 2021.
- [46] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

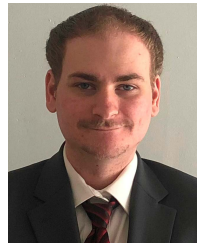
- [47] P. Adarsh, P. Rathi, and M. Kumar, "YOLO v3-Tiny: Object detection and recognition using one stage improved model," in *Proc. 6th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Mar. 2020, pp. 687–694.



Mohammed E. Elbity (Student Member, IEEE) received the B.Sc. degree in electronics and communication engineering from Al-Azhar University, Egypt, in 2017. He is currently pursuing the Ph.D. degree with the UofSC, USA. After graduation, he joined the Nanoelectronics Integrated Systems Center (NISC), Egypt, as a Research Assistant, where he worked on emerging memory technologies and memristor-based architectures. In 2019, he joined the Mixed-Signal Integrated Systems Laboratory, South Korea, and he worked on touch-screen controllers and AI hardware accelerators. He is also a Research Assistant at the iCAS Laboratory. His current research interests include in-memory computing, emerging memory technologies, blockchain hardware acceleration, and intelligent systems.



Peyton S. Chandarana received the B.S.E. degree in computer engineering from the University of South Carolina in 2019, where he is currently pursuing the Ph.D. degree with the Intelligent Circuits, Architectures, and Systems (iCAS) Laboratory, under Dr. Ramtin Zand. His current research interests include neuromorphic computing topics in both algorithm and hardware design.



Brendan Reidy is currently pursuing the master's degree in computer science with the iCAS Laboratory, University of South Carolina. He is also working as a Research Assistant with the iCAS Laboratory, University of South Carolina. His current research interests include computer vision, natural language processing, and model compression and optimization for implementation on edge devices.



Jason K. Eshraghian (Member, IEEE) received the B.Eng. (Electrical and Electronic), L.L.B., and Ph.D. degrees from The University of Western Australia, Perth, WA, Australia, in 2017 and 2019, respectively. From 2019 to 2022, he was a Post-Doctoral Research Fellow at the University of Michigan, Ann Arbor MI, USA. He is currently an Assistant Professor with the Department of Electrical and Computer Engineering, University of California at Santa Cruz, Santa Cruz, CA, USA. His research interests include neuromorphic computing, resistive random access memory (RRAM) circuits, and spiking neural networks.



Ramtin Zand received the Ph.D. degree in computer engineering from the University of Central Florida (UCF), Orlando, FL, USA, in 2019. He is currently an Assistant Professor of computer science and engineering at the University of South Carolina (UofSC), and the Director of the Intelligent Circuits, Architectures, and Systems (iCAS) Laboratory. His research interests include hardware accelerator design for data-intensive and compute-intensive applications, in-memory computing, and neuromorphic computing.